

Analyzing Customer Orders Using Python - Course-End Project 1

Student: Guilherme Couto

Email: guimcouto76@gmail.com

Program: Professional Certificate in AI and Machine Learning

Institution: Purdue University

Cohort: Purdue PCP AIML Nov 2025 – Cohort 89

Project: Analyzing Customer Orders Using Python (Course-End Project 1)

I kept the same task numbering as the project statement so each requirement is easy to find.

Project goal: Use Python's built-in data structures (no pandas, no external libraries) to classify products, look at customer purchasing patterns, and pull out a few business insights from a small set of customer orders.

Dataset: The brief says no dataset is provided, so I built one in Python directly. I kept it small enough to read but added repeated products across customers so the frequency analysis later in the notebook is not trivial.

Table of Contents

1. Store customer orders (1.1 customer list, 1.2 order tuples, 1.3 customer-to-products dictionary)
2. Classify products by category (2.1 product-category map, 2.2 unique category set, 2.3 display categories)
3. Analyze customer orders (3.1 per-customer spending, 3.2 high / moderate / low classification)
4. Generate business insights (4.1 revenue by category, 4.2 unique products set, 4.3 electronics customers, 4.4 top 3 spenders, 4.5 product frequency and high-value customers)
5. Organize and display data (5.1 customer summary, 5.2 multi-category customers, 5.3 customers in both Electronics and Clothing, 5.4 retrieve and modify dictionary records, 5.5 product set operations across categories)
6. Final Report and Business Recommendations
7. Conclusion

1. Store customer orders

1.1 Create a list of customer names

```
In [1]: customer_names = [
        "Alice Johnson", "Brian Smith", "Carla Mendes", "Daniel Brown", "Emma Wilson",
        "Felipe Costa", "Grace Lee", "Henry Davis", "Isabella Moore", "Jack Taylor"
    ]

    print("Customer Names:")
    for customer in customer_names:
        print("-", customer)
```

Customer Names:

- Alice Johnson
- Brian Smith
- Carla Mendes
- Daniel Brown
- Emma Wilson
- Felipe Costa
- Grace Lee
- Henry Davis
- Isabella Moore
- Jack Taylor

1.2 Store each customer's order details as tuples inside a list

Each order is a tuple (customer_name, product, price, category). I used tuples instead of lists because an order should not be modified after it is recorded.

```
In [2]: customer_orders = [
        ("Alice Johnson", "Laptop", 950.00, "Electronics"),
        ("Alice Johnson", "Wireless Mouse", 25.00, "Electronics"),
        ("Alice Johnson", "Headphones", 120.00, "Electronics"),
        ("Alice Johnson", "Keyboard", 70.00, "Electronics"),
        ("Alice Johnson", "Desk Lamp", 35.00, "Home Essentials"),
        ("Alice Johnson", "Notebook", 12.00, "Office Supplies"),
        ("Alice Johnson", "Water Bottle", 18.00, "Accessories"),

        ("Brian Smith", "T-Shirt", 20.00, "Clothing"),
        ("Brian Smith", "Jeans", 45.00, "Clothing"),
        ("Brian Smith", "Pen Set", 15.00, "Office Supplies"),
        ("Brian Smith", "Water Bottle", 18.00, "Accessories"),

        ("Carla Mendes", "Coffee Maker", 80.00, "Home Essentials"),
        ("Carla Mendes", "Headphones", 120.00, "Electronics"),
        ("Carla Mendes", "Vacuum Cleaner", 150.00, "Home Essentials"),
        ("Carla Mendes", "Bluetooth Speaker", 85.00, "Electronics"),
        ("Carla Mendes", "T-Shirt", 20.00, "Clothing"),
        ("Carla Mendes", "Water Bottle", 18.00, "Accessories"),

        ("Daniel Brown", "Notebook", 12.00, "Office Supplies"),
        ("Daniel Brown", "Socks", 10.00, "Clothing"),
        ("Daniel Brown", "Water Bottle", 18.00, "Accessories"),

        ("Emma Wilson", "Smartphone", 700.00, "Electronics"),
        ("Emma Wilson", "Jacket", 90.00, "Clothing"),
        ("Emma Wilson", "Bluetooth Speaker", 85.00, "Electronics"),
        ("Emma Wilson", "Laptop", 950.00, "Electronics"),
        ("Emma Wilson", "T-Shirt", 20.00, "Clothing"),
```

```

("Emma Wilson", "Vacuum Cleaner", 150.00, "Home Essentials"),

("Felipe Costa", "Blender", 60.00, "Home Essentials"),
("Felipe Costa", "Running Shoes", 110.00, "Clothing"),
("Felipe Costa", "Microwave", 140.00, "Home Essentials"),
("Felipe Costa", "Smartphone", 700.00, "Electronics"),
("Felipe Costa", "Coffee Maker", 80.00, "Home Essentials"),
("Felipe Costa", "Jeans", 45.00, "Clothing"),
("Felipe Costa", "Keyboard", 70.00, "Electronics"),

("Grace Lee", "Tablet", 450.00, "Electronics"),
("Grace Lee", "Backpack", 55.00, "Accessories"),
("Grace Lee", "Laptop", 950.00, "Electronics"),
("Grace Lee", "Jacket", 90.00, "Clothing"),
("Grace Lee", "Dress", 75.00, "Clothing"),
("Grace Lee", "Pillow", 30.00, "Home Essentials"),
("Grace Lee", "Sunglasses", 95.00, "Accessories"),

("Henry Davis", "Pillow", 30.00, "Home Essentials"),
("Henry Davis", "Bedsheet Set", 65.00, "Home Essentials"),
("Henry Davis", "Coffee Maker", 80.00, "Home Essentials"),
("Henry Davis", "Headphones", 120.00, "Electronics"),

("Isabella Moore", "Smartwatch", 220.00, "Electronics"),
("Isabella Moore", "Dress", 75.00, "Clothing"),
("Isabella Moore", "Tablet", 450.00, "Electronics"),
("Isabella Moore", "Running Shoes", 110.00, "Clothing"),
("Isabella Moore", "Headphones", 120.00, "Electronics"),
("Isabella Moore", "Bedsheet Set", 65.00, "Home Essentials"),
("Isabella Moore", "Sunglasses", 95.00, "Accessories"),

("Jack Taylor", "Water Bottle", 18.00, "Accessories"),
("Jack Taylor", "Cap", 22.00, "Clothing"),
("Jack Taylor", "T-Shirt", 20.00, "Clothing"),
("Jack Taylor", "Socks", 10.00, "Clothing"),
("Jack Taylor", "Notebook", 12.00, "Office Supplies"),
]

print(f"Total orders: {len(customer_orders)}")
print("\nSample Customer Orders (first 10):")
for order in customer_orders[:10]:
    print(order)

```

Total orders: 56

Sample Customer Orders (first 10):

```

('Alice Johnson', 'Laptop', 950.0, 'Electronics')
('Alice Johnson', 'Wireless Mouse', 25.0, 'Electronics')
('Alice Johnson', 'Headphones', 120.0, 'Electronics')
('Alice Johnson', 'Keyboard', 70.0, 'Electronics')
('Alice Johnson', 'Desk Lamp', 35.0, 'Home Essentials')
('Alice Johnson', 'Notebook', 12.0, 'Office Supplies')
('Alice Johnson', 'Water Bottle', 18.0, 'Accessories')
('Brian Smith', 'T-Shirt', 20.0, 'Clothing')
('Brian Smith', 'Jeans', 45.0, 'Clothing')
('Brian Smith', 'Pen Set', 15.0, 'Office Supplies')

```

1.3 Use a dictionary where keys are customer names and values are lists of ordered products

```
In [3]: customer_product_dictionary = {}

# setdefault avoids a "key not in dict" check each iteration
for customer, product, price, category in customer_orders:
    customer_product_dictionary.setdefault(customer, []).append(product)

for customer, products in customer_product_dictionary.items():
    print(f"{customer}: {products}")
```

```
Alice Johnson: ['Laptop', 'Wireless Mouse', 'Headphones', 'Keyboard', 'Desk Lamp', 'Notebook', 'Water Bottle']
Brian Smith: ['T-Shirt', 'Jeans', 'Pen Set', 'Water Bottle']
Carla Mendes: ['Coffee Maker', 'Headphones', 'Vacuum Cleaner', 'Bluetooth Speaker', 'T-Shirt', 'Water Bottle']
Daniel Brown: ['Notebook', 'Socks', 'Water Bottle']
Emma Wilson: ['Smartphone', 'Jacket', 'Bluetooth Speaker', 'Laptop', 'T-Shirt', 'Vacuum Cleaner']
Felipe Costa: ['Blender', 'Running Shoes', 'Microwave', 'Smartphone', 'Coffee Maker', 'Jeans', 'Keyboard']
Grace Lee: ['Tablet', 'Backpack', 'Laptop', 'Jacket', 'Dress', 'Pillow', 'Sunglasses']
Henry Davis: ['Pillow', 'Bedsheet Set', 'Coffee Maker', 'Headphones']
Isabella Moore: ['Smartwatch', 'Dress', 'Tablet', 'Running Shoes', 'Headphones', 'Bedsheet Set', 'Sunglasses']
Jack Taylor: ['Water Bottle', 'Cap', 'T-Shirt', 'Socks', 'Notebook']
```

1. Response

I stored the data in three pieces. A flat list for the customer names. A list of 4-tuples for the orders, where each tuple holds the customer, product, price, and category. And a dictionary keyed by customer name that points to the list of products that customer ordered, which makes per-customer lookups quick.

I used `dict.setdefault` to build the customer-to-products dictionary so I did not have to check whether the key already existed on every iteration.

2. Classify products by category

2.1 Use a dictionary to map each product to its respective category

```
In [4]: product_category_map = {}

for customer, product, price, category in customer_orders:
    product_category_map[product] = category

for product, category in product_category_map.items():
    print(f"{product}: {category}")
```

Laptop: Electronics
Wireless Mouse: Electronics
Headphones: Electronics
Keyboard: Electronics
Desk Lamp: Home Essentials
Notebook: Office Supplies
Water Bottle: Accessories
T-Shirt: Clothing
Jeans: Clothing
Pen Set: Office Supplies
Coffee Maker: Home Essentials
Vacuum Cleaner: Home Essentials
Bluetooth Speaker: Electronics
Socks: Clothing
Smartphone: Electronics
Jacket: Clothing
Blender: Home Essentials
Running Shoes: Clothing
Microwave: Home Essentials
Tablet: Electronics
Backpack: Accessories
Dress: Clothing
Pillow: Home Essentials
Sunglasses: Accessories
Bedsheet Set: Home Essentials
Smartwatch: Electronics
Cap: Clothing

2.2 Create a set of unique product categories

```
In [5]: unique_categories = set(product_category_map.values())  
        print(unique_categories)  
  
{'Electronics', 'Accessories', 'Clothing', 'Office Supplies', 'Home Essentials'}
```

2.3 Display all available product categories

```
In [6]: print("Available Product Categories:")  
        for category in sorted(unique_categories):  
            print("-", category)
```

Available Product Categories:
- Accessories
- Clothing
- Electronics
- Home Essentials
- Office Supplies

2. Response

The dataset has five product categories: Accessories, Clothing, Electronics, Home Essentials, and Office Supplies. The dictionary gives me a fast product-to-category lookup, and passing the values into `set()` removed the duplicates without an extra loop.

3. Analyze customer orders

3.1 Use a loop to calculate the total amount each customer spends

```
In [7]: customer_spending = {}

for customer, product, price, category in customer_orders:
    customer_spending[customer] = customer_spending.get(customer, 0) + price

for customer, total in customer_spending.items():
    print(f"{customer}: ${total:.2f}")
```

Alice Johnson: \$1230.00
Brian Smith: \$98.00
Carla Mendes: \$473.00
Daniel Brown: \$40.00
Emma Wilson: \$1995.00
Felipe Costa: \$1205.00
Grace Lee: \$1745.00
Henry Davis: \$295.00
Isabella Moore: \$1135.00
Jack Taylor: \$82.00

3.2 Classify customers based on total purchase value

- Above \$100: High-value buyer
- Between \$50 and \$100: Moderate buyer
- Below \$50: Low-value buyer

I read "between \$50 and \$100" as inclusive on both ends, so the rules become `total > 100`, `50 <= total <= 100`, and `total < 50`.

```
In [8]: customer_classification = {}

# > 100 high, [50, 100] moderate, < 50 low
for customer, total in customer_spending.items():
    if total > 100:
        classification = "High-value buyer"
    elif 50 <= total <= 100:
        classification = "Moderate buyer"
    else:
        classification = "Low-value buyer"
    customer_classification[customer] = classification

for customer, classification in customer_classification.items():
    print(f"{customer}: {classification}")
```

Alice Johnson: High-value buyer
Brian Smith: Moderate buyer
Carla Mendes: High-value buyer
Daniel Brown: Low-value buyer
Emma Wilson: High-value buyer
Felipe Costa: High-value buyer
Grace Lee: High-value buyer
Henry Davis: High-value buyer
Isabella Moore: High-value buyer
Jack Taylor: Moderate buyer

3. Response

Seven of the ten customers came out as high-value buyers. That makes sense once you look at the totals: a single Electronics purchase like a laptop or a smartphone is enough to push a customer past \$100. The remaining three (Brian, Jack, Daniel) only bought cheaper Clothing, Accessories, and Office Supplies items. This split is useful for marketing because it lets us send different offers to each segment.

4. Generate business insights

4.1 Calculate the total revenue per product category and store it in a dictionary

```
In [9]: category_revenue = {}

for customer, product, price, category in customer_orders:
    category_revenue[category] = category_revenue.get(category, 0) + price

for category, revenue in sorted(category_revenue.items(), key=lambda x: x[1], reverse=True):
    print(f"{category}: ${revenue:.2f}")
```

Electronics: \$6185.00
Home Essentials: \$965.00
Clothing: \$762.00
Accessories: \$335.00
Office Supplies: \$51.00

4.2 Extract unique products from all orders using a set

```
In [10]: unique_products = {product for customer, product, price, category in customer_orders}

for product in sorted(unique_products):
    print("-", product)
```

- Backpack
- Bedsheet Set
- Blender
- Bluetooth Speaker
- Cap
- Coffee Maker
- Desk Lamp
- Dress
- Headphones
- Jacket
- Jeans
- Keyboard
- Laptop
- Microwave
- Notebook
- Pen Set
- Pillow
- Running Shoes
- Smartphone
- Smartwatch
- Socks
- Sunglasses
- T-Shirt
- Tablet
- Vacuum Cleaner
- Water Bottle
- Wireless Mouse

4.3 Use a list comprehension to find all customers who purchased electronics

```
In [11]: # List comprehension first, then dedup with set() and sort
electronics_customers_all = [
    customer
    for customer, product, price, category in customer_orders
    if category == "Electronics"
]

electronics_customers = sorted(set(electronics_customers_all))

print("Raw list comprehension result (may contain duplicates):")
print(electronics_customers_all)

print("\nUnique customers who purchased Electronics:")
for customer in electronics_customers:
    print("-", customer)
```


Raw list comprehension result (may contain duplicates):

```
['Alice Johnson', 'Alice Johnson', 'Alice Johnson', 'Alice Johnson', 'Carla Mend  
s', 'Carla Mendes', 'Emma Wilson', 'Emma Wilson', 'Emma Wilson', 'Felipe Costa',  
'Felipe Costa', 'Grace Lee', 'Grace Lee', 'Henry Davis', 'Isabella Moore', 'Isabe  
lla Moore', 'Isabella Moore']
```

Unique customers who purchased Electronics:

- Alice Johnson
- Carla Mendes
- Emma Wilson
- Felipe Costa
- Grace Lee
- Henry Davis
- Isabella Moore

4.4 Identify the top three highest-spending customers using sorting

```
In [12]: top_three_customers = sorted(  
        customer_spending.items(),  
        key=lambda item: item[1],  
        reverse=True  
)[:3]  
  
for rank, (customer, total) in enumerate(top_three_customers, start=1):  
    print(f"{rank}. {customer}: ${total:.2f}")
```

1. Emma Wilson: \$1995.00
2. Grace Lee: \$1745.00
3. Alice Johnson: \$1230.00

4.5 Extract the high-value customers and most frequently purchased products

```
In [13]: high_value_customers = [  
        customer  
        for customer, classification in customer_classification.items()  
        if classification == "High-value buyer"  
]  
  
print("High-Value Customers:")  
for customer in high_value_customers:  
    print("-", customer)  
  
product_frequency = {}  
for customer, product, price, category in customer_orders:  
    product_frequency[product] = product_frequency.get(product, 0) + 1  
  
sorted_frequency = sorted(  
    product_frequency.items(),  
    key=lambda item: (-item[1], item[0])  
)  
  
print("\nMost Frequently Purchased Products (top 10):")  
print(f"{'Product':<20}{'Times Purchased':>16}")  
print("-" * 36)  
for product, count in sorted_frequency[:10]:
```

```

print(f"{product:<20}{count:>16}")

repeat_products = [(p, c) for p, c in sorted_frequency if c > 1]
print(f"\nProducts purchased by more than one customer: {len(repeat_products)}")
print(f"Products purchased only once: {len(sorted_frequency) - len(repeat_products)}")

```

High-Value Customers:

- Alice Johnson
- Carla Mendes
- Emma Wilson
- Felipe Costa
- Grace Lee
- Henry Davis
- Isabella Moore

Most Frequently Purchased Products (top 10):

Product	Times Purchased
Water Bottle	5
Headphones	4
T-Shirt	4
Coffee Maker	3
Laptop	3
Notebook	3
Bedsheet Set	2
Bluetooth Speaker	2
Dress	2
Jacket	2

Products purchased by more than one customer: 19

Products purchased only once: 8

4. Response

Electronics is the strongest category by revenue, mostly because of high-priced items like laptops, smartphones, and tablets. Home Essentials comes in second thanks to vacuums, microwaves, and coffee makers. Sorting the spending dictionary and slicing the first three entries is a clean way to get the top spenders without tracking maxes manually.

5. Organize and display data

5.1 Print a summary of each customer's total spending and their classification

```

In [14]: header = f"{'Customer':<18}{'Total Spending':>16}{'Classification':>20} Product
print("Customer Spending Summary")
print("=" * len(header))
print(header)
print("-" * len(header))

for customer in sorted(customer_spending):
    total = customer_spending[customer]
    classification = customer_classification[customer]
    products = ", ".join(customer_product_dictionary[customer])
    amount = f"${total:.2f}"

```

```
print(f"{customer:<18}{amount:>16}{classification:>20} {products}")

print("=" * len(header))
```

Customer Spending Summary

```
=====
Customer          Total Spending      Classification  Products Ordered
-----
Alice Johnson      $1230.00      High-value buyer  Laptop, Wireless Mouse, H
eadphones, Keyboard, Desk Lamp, Notebook, Water Bottle
Brian Smith        $98.00        Moderate buyer   T-Shirt, Jeans, Pen Set,
Water Bottle
Carla Mendes       $473.00      High-value buyer  Coffee Maker, Headphones,
Vacuum Cleaner, Bluetooth Speaker, T-Shirt, Water Bottle
Daniel Brown       $40.00        Low-value buyer   Notebook, Socks, Water Bo
ttle
Emma Wilson        $1995.00      High-value buyer  Smartphone, Jacket, Bluet
ooth Speaker, Laptop, T-Shirt, Vacuum Cleaner
Felipe Costa       $1205.00      High-value buyer  Blender, Running Shoes, M
icrowave, Smartphone, Coffee Maker, Jeans, Keyboard
Grace Lee          $1745.00      High-value buyer  Tablet, Backpack, Laptop,
Jacket, Dress, Pillow, Sunglasses
Henry Davis        $295.00      High-value buyer  Pillow, Bedsheet Set, Cof
fee Maker, Headphones
Isabella Moore     $1135.00      High-value buyer  Smartwatch, Dress, Table
t, Running Shoes, Headphones, Bedsheet Set, Sunglasses
Jack Taylor        $82.00        Moderate buyer   Water Bottle, Cap, T-Shir
t, Socks, Notebook
=====
```

5.2 Use set operations to find customers who purchased from multiple categories

```
In [15]: customer_categories = {}

for customer, product, price, category in customer_orders:
    customer_categories.setdefault(customer, set()).add(category)

multiple_category_customers = {
    customer
    for customer, categories in customer_categories.items()
    if len(categories) > 1
}

for customer in sorted(multiple_category_customers):
    print(f"{customer}: {customer_categories[customer]}")
```

```
Alice Johnson: {'Electronics', 'Accessories', 'Office Supplies', 'Home Essentials'}
Brian Smith: {'Office Supplies', 'Clothing', 'Accessories'}
Carla Mendes: {'Electronics', 'Accessories', 'Clothing', 'Home Essentials'}
Daniel Brown: {'Accessories', 'Office Supplies', 'Clothing'}
Emma Wilson: {'Electronics', 'Clothing', 'Home Essentials'}
Felipe Costa: {'Electronics', 'Clothing', 'Home Essentials'}
Grace Lee: {'Electronics', 'Accessories', 'Clothing', 'Home Essentials'}
Henry Davis: {'Electronics', 'Home Essentials'}
Isabella Moore: {'Electronics', 'Accessories', 'Clothing', 'Home Essentials'}
Jack Taylor: {'Accessories', 'Clothing', 'Office Supplies'}
```

5.3 Identify common customers who bought both electronics and clothing

```
In [16]: electronics_customer_set = {
    customer
    for customer, product, price, category in customer_orders
    if category == "Electronics"
}

clothing_customer_set = {
    customer
    for customer, product, price, category in customer_orders
    if category == "Clothing"
}

electronics_and_clothing_customers = electronics_customer_set.intersection(clothing_customer_set)

print("Customers Who Bought Both Electronics and Clothing:")
for customer in sorted(electronics_and_clothing_customers):
    print("-", customer)

print("\nElectronics Customers:", electronics_customer_set)
print("Clothing Customers:", clothing_customer_set)
print("Intersection:", electronics_and_clothing_customers)
```

Customers Who Bought Both Electronics and Clothing:

- Carla Mendes
- Emma Wilson
- Felipe Costa
- Grace Lee
- Isabella Moore

Electronics Customers: {'Emma Wilson', 'Carla Mendes', 'Alice Johnson', 'Felipe Costa', 'Isabella Moore', 'Grace Lee', 'Henry Davis'}

Clothing Customers: {'Emma Wilson', 'Carla Mendes', 'Brian Smith', 'Jack Taylor', 'Daniel Brown', 'Felipe Costa', 'Isabella Moore', 'Grace Lee'}

Intersection: {'Emma Wilson', 'Carla Mendes', 'Felipe Costa', 'Isabella Moore', 'Grace Lee'}

5.4 Retrieve and modify customer records using dictionary methods

```
In [17]: # work on a copy so the demo does not mutate the dict used in the final report
demo_products = {
    customer: list(products)
    for customer, products in customer_product_dictionary.items()
}

selected_customer = "Alice Johnson"

alice_products = demo_products.get(selected_customer, [])
print(f"Original products for {selected_customer}: {alice_products}")

demo_products.setdefault(selected_customer, []).append("USB-C Cable")
print(f"Updated products for {selected_customer} (demo copy): {demo_products[selected_customer]}")
```

```
print(f"\nOriginal dictionary unchanged for {selected_customer}: "
      f"{customer_product_dictionary[selected_customer]}")

new_customer = "Karen Patel"
demo_products.setdefault(new_customer, []).append("Gift Card")
print(f"Added new customer record in demo copy -> {new_customer}: {demo_products
```

Original products for Alice Johnson: ['Laptop', 'Wireless Mouse', 'Headphones', 'Keyboard', 'Desk Lamp', 'Notebook', 'Water Bottle']

Updated products for Alice Johnson (demo copy): ['Laptop', 'Wireless Mouse', 'Headphones', 'Keyboard', 'Desk Lamp', 'Notebook', 'Water Bottle', 'USB-C Cable']

Original dictionary unchanged for Alice Johnson: ['Laptop', 'Wireless Mouse', 'Headphones', 'Keyboard', 'Desk Lamp', 'Notebook', 'Water Bottle']

Added new customer record in demo copy -> Karen Patel: ['Gift Card']

5.5 Use set operations to find common and unique products across categories

The brief asks for set operations across categories. Since each product belongs to exactly one category, the intersection of any two category-product sets is empty, which is what the output below shows. The same cell also prints the products unique to each category and the union of products across Electronics, Clothing, and Home Essentials.

```
In [18]: products_by_category = {}
for customer, product, price, category in customer_orders:
    products_by_category.setdefault(category, set()).add(product)

print("Products per category (set per category):")
for category in sorted(products_by_category):
    print(f" {category}: {sorted(products_by_category[category])}")

electronics_products = products_by_category.get("Electronics", set())
clothing_products     = products_by_category.get("Clothing", set())
home_products         = products_by_category.get("Home Essentials", set())

unique_to_electronics = electronics_products - clothing_products - home_products
unique_to_clothing     = clothing_products - electronics_products - home_products
electronics_and_clothing_products = electronics_products & clothing_products
all_three_categories_union = electronics_products | clothing_products | home_products

print("\nUnique to Electronics (set difference):")
print(sorted(unique_to_electronics))

print("\nUnique to Clothing (set difference):")
print(sorted(unique_to_clothing))

print("\nProducts that appear in BOTH Electronics and Clothing (intersection):")
print(electronics_and_clothing_products or "(empty - categories are disjoint by

print("\nUnion of products in Electronics, Clothing and Home Essentials:")
print(f" {len(all_three_categories_union)} unique products")
print(f" {sorted(all_three_categories_union)}")
```

Products per category (set per category):

```
Accessories: ['Backpack', 'Sunglasses', 'Water Bottle']
Clothing: ['Cap', 'Dress', 'Jacket', 'Jeans', 'Running Shoes', 'Socks', 'T-Shirt']
Electronics: ['Bluetooth Speaker', 'Headphones', 'Keyboard', 'Laptop', 'Smartphone', 'Smartwatch', 'Tablet', 'Wireless Mouse']
Home Essentials: ['Bedsheet Set', 'Blender', 'Coffee Maker', 'Desk Lamp', 'Microwave', 'Pillow', 'Vacuum Cleaner']
Office Supplies: ['Notebook', 'Pen Set']
```

Unique to Electronics (set difference):

```
['Bluetooth Speaker', 'Headphones', 'Keyboard', 'Laptop', 'Smartphone', 'Smartwatch', 'Tablet', 'Wireless Mouse']
```

Unique to Clothing (set difference):

```
['Cap', 'Dress', 'Jacket', 'Jeans', 'Running Shoes', 'Socks', 'T-Shirt']
```

Products that appear in BOTH Electronics and Clothing (intersection):

(empty - categories are disjoint by design)

Union of products in Electronics, Clothing and Home Essentials:

22 unique products

```
['Bedsheet Set', 'Blender', 'Bluetooth Speaker', 'Cap', 'Coffee Maker', 'Desk Lamp', 'Dress', 'Headphones', 'Jacket', 'Jeans', 'Keyboard', 'Laptop', 'Microwave', 'Pillow', 'Running Shoes', 'Smartphone', 'Smartwatch', 'Socks', 'T-Shirt', 'Tablet', 'Vacuum Cleaner', 'Wireless Mouse']
```

5. Response

Putting the totals, classifications, and product lists into one summary made the patterns easier to read. The set operations in 5.2 and 5.3 turned customer segmentation into one-line expressions: `len(categories) > 1` for multi-category buyers and `&` for the Electronics-and-Clothing overlap. In 5.4 I worked on a copy of the customer-to-products dictionary so the demo modifications would not affect the final report.

Final Report and Business Recommendations

Customer Classification Summary

```
In [19]: classification_summary = []

for customer, total in customer_spending.items():
    classification_summary.append({
        "Customer": customer,
        "Total Spending": total,
        "Classification": customer_classification[customer],
        "Number of Categories Purchased": len(customer_categories[customer])
    })

classification_summary = sorted(
    classification_summary,
    key=lambda item: item["Total Spending"],
    reverse=True
```

```

)

header = f"{'Rank':>4} {'Customer':<18}{'Spending':>14}{'Classification':>22}{'Number of Categories Purchased':>8}"
print(header)
print("-" * len(header))

for rank, row in enumerate(classification_summary, start=1):
    amount = f"${row['Total Spending']:.2f}"
    print(
        f"{rank:>4} "
        f"{row['Customer']:<18}"
        f"{amount:>14}"
        f"{row['Classification']:>22}"
        f"{row['Number of Categories Purchased']:>8}"
    )

print("-" * len(header))

total_revenue = sum(customer_spending.values())
high_count = sum(1 for r in classification_summary if r["Classification"] == "High-value buyer")
mod_count = sum(1 for r in classification_summary if r["Classification"] == "Moderate buyer")
low_count = sum(1 for r in classification_summary if r["Classification"] == "Low-value buyer")

print(f"\nTotal customers: {len(classification_summary)}")
print(f"Total revenue: ${total_revenue:.2f}")
print(f"High-value buyers: {high_count}")
print(f"Moderate buyers: {mod_count}")
print(f"Low-value buyers: {low_count}")

```

Rank	Customer	Spending	Classification	#Cats

1	Emma Wilson	\$1995.00	High-value buyer	3
2	Grace Lee	\$1745.00	High-value buyer	4
3	Alice Johnson	\$1230.00	High-value buyer	4
4	Felipe Costa	\$1205.00	High-value buyer	3
5	Isabella Moore	\$1135.00	High-value buyer	4
6	Carla Mendes	\$473.00	High-value buyer	4
7	Henry Davis	\$295.00	High-value buyer	2
8	Brian Smith	\$98.00	Moderate buyer	3
9	Jack Taylor	\$82.00	Moderate buyer	3
10	Daniel Brown	\$40.00	Low-value buyer	3

```

Total customers: 10
Total revenue: $8298.00
High-value buyers: 7
Moderate buyers: 2
Low-value buyers: 1

```

Final Business Insights

A few things stand out from the numbers and are worth passing along to the business team:

1. Electronics is the strongest category, accounting for about 75% of total revenue. It is the obvious area to focus inventory and promotions on.

2. Home Essentials is a solid second category. Items like vacuum cleaners, microwaves, and coffee makers add up quickly, so a seasonal promotion could increase the average order value.
3. Seven of the ten customers are high-value buyers and they generate almost all of the revenue. Loyalty perks or exclusive discounts would help retain them.
4. The two moderate buyers (Brian and Jack) spend close to \$100. A small incentive such as free shipping above a threshold could move them into the high-value group.
5. The one low-value buyer (Daniel) only spent \$40, all on cheaper items. A targeted coupon or a small recommendation email is a low-cost way to try to re-engage him.
6. Most customers shop across multiple categories already, so cross-category campaigns are likely to work well for this audience.
7. Five customers bought both Electronics and Clothing. They are a clear target for bundle offers such as smartwatch + running shoes or tablet + backpack.

Conclusion

This project used only Python's standard library to classify customers, compute revenue by category, identify cross-category buyers, and produce a short list of business recommendations.

Each data structure had a clear job. Lists held ordered data such as the customer names and the orders. Tuples represented individual orders that should not change after creation. Dictionaries gave fast lookups by customer or by product. Sets handled deduplication and the category overlap analysis. The control flow was simple: loops, conditionals, sorting, a few list and set comprehensions, and basic set operations.

For a dataset of this size, the standard library is enough. Working without pandas also made me think more carefully about which structure to use for each step, rather than putting everything into a DataFrame by default.